

Training Conclusion: 15-Class Attack Classifier

How the multi-class XGBoost model was trained, how its trees are stored, what data types it uses, how it was tested, and what every result means. The model labels each network flow as Benign or one of 14 attack types.

Overall accuracy 95.7% 445,806 of 465,837 correct	Classes at 95%+ 10 / 15 recall on the test set	Weighted F1 0.956 averaged by class size	Macro F1 0.792 every class counts equally	Trees 7,500 7,470 used to predict
--	---	---	--	--

Verdict. The overall target is met: 95.7% of test flows are labelled correctly. 10 of the 15 classes reach 95% recall (95 of every 100 attacks caught). Five do not: Infiltration, Brute Force -XSS, SQL Injection, FTP-BruteForce and DoS attacks-SlowHTTPTest. Section 07 explains why.

01 — THE DATA

What the model learned from

Source: **cicids2018_full_cleaned.csv** (11,872,582 cleaned flows, 79 columns). All attack rows are used. Benign has 10.5M rows, more than this 12 GB machine can train on, so a random sample of **999,610** Benign rows is used. Working set: **2,329,184 rows**.

Each class is split on its own: **80% for training, 20% for testing** (a stratified split). From the training 80%, a further 10% is held back as a **validation** set that decides when training stops and tunes the class weights. The test set is never seen during training or tuning.

Class (label number)	Rows used	Training (80%)	Test (20%)
Benign (0)	999,610	799,688	199,922
Bot (1)	144,535	115,628	28,907
Brute Force -Web (2)	547	438	109
Brute Force -XSS (3)	226	181	45
DDOS attack-HOIC (4)	198,861	159,089	39,772
DDOS attack-LOIC-UDP (5)	1,730	1,384	346
DDoS attacks-LOIC-HTTP (6)	575,364	460,291	115,073
DoS attacks-GoldenEye (7)	41,406	33,125	8,281
DoS attacks-Hulk (8)	145,199	116,159	29,040
DoS attacks-SlowHTTPTest (9)	43	34	9
DoS attacks-Slowloris (10)	9,908	7,926	1,982
FTP-BruteForce (11)	39	31	8
Infiltration (12)	117,598	94,078	23,520
SQL Injection (13)	77	62	15
SSH-Bruteforce (14)	94,041	75,233	18,808
Total	2,329,184	1,863,347	465,837

The training 80% (1,863,347 rows) splits into 1,677,012 rows the trees learn from and 186,335 validation rows. Label numbers are assigned alphabetically by the label encoder; label_classes.json maps them back to names.

02 — HOW IT WAS TRAINED

Gradient-boosted decision trees, step by step

1. **Load in chunks.** The 4.8 GB CSV is read 200,000 rows at a time as float32. All attack rows and a random Benign sample are kept.
2. **Encode labels.** The 15 class names become the numbers 0–14.
3. **Choose features.** 78 feature columns minus the 8 columns that are zero in every row = **70 features**.
4. **Split.** Stratified 80/20 train/test, then 10% of train kept for validation.
5. **Weight the classes.** Each training row gets a weight so rare attacks are not drowned out by common ones. The weight is the square root of the standard "balanced" weight, which lifts rare classes gently.
6. **Grow trees.** XGBoost adds trees in rounds. Each round builds **15 trees, one per class**; each tree corrects the errors of the rounds before it for its class.
7. **Stop at the best point.** After every round the model is scored on the validation set (multi-class log-loss). Training ran all 500 rounds; the best score (0.1148) was at round 497, so prediction uses rounds 0–497. Training took about 16 minutes.
8. **Tune class weights.** A second script searches one decision weight per class on the validation set (section 04).

Setting	Value	What it does
objective	multi:softprob	outputs 15 probabilities that add up to 1
num_class	15	Benign + 14 attack types
n_estimators	500	maximum boosting rounds (each round = 15 trees)
learning_rate	0.1	how much each new tree is allowed to change the prediction
max_depth	10	deepest a tree may grow: at most 10 yes/no questions per path
subsample	0.8	each tree sees a random 80% of the training rows
colsample_bytree	0.8	each tree sees a random 80% of the 70 features
tree_method	hist	features are bucketed into histograms for fast training
early_stopping_rounds	40	stop if validation loss fails to improve for 40 rounds
eval_metric	mlogloss	multi-class log-loss on the validation set (unweighted)
sample_weights	sqrt(balanced)	rare classes weighted up, softened by a square root
random_state	42	fixed seed, so the run can be reproduced

How one prediction is made. Each of the 15 classes gets a score: its starting value (base_score) plus the output of its 498 trees. The softmax function turns the 15 scores into 15 probabilities that sum to 1. The class with the highest weighted probability is the answer.

```
score[c] = base_score[c] + sum of the leaf values of class c's 498 trees      (c = 0..14)
prob[c]  = exp(score[c]) / sum over all 15 classes of exp(score)           (softmax)
answer   = the class c with the largest prob[c] * class_weight[c]
```

What did not work first. The first run used full "balanced" weights and stopped on the weighted validation loss. It stopped after only 22 rounds and reached 85.5% overall, with Benign recall falling to 68%. Softening the weights and stopping on the unweighted loss raised overall accuracy to 95.7%.

03 — HOW THE TREES ARE STORED

Inside all_attacks_xgb.json

The trained model is one **JSON text file** (55.6 MB), written by XGBoost 3.4.1 with `save_model()`. JSON is XGBoost's portable format: XGBoost 3.4.1 or newer can load it on any operating system, and it can be opened in a text editor.

```
all_attacks_xgb.json
|-- version           [3, 4, 1]
`-- learner
    |-- objective      multi:softprob
    |-- learner_model_param  num_class 15, num_feature 70, base_score (15 numbers)
    |-- feature_names / types  the 70 column names, all 'float'
    |-- attributes      best_iteration 497, best_score 0.1148
    `-- gradient_booster (gbtree)
        |-- model
            |-- tree_info  7,500 numbers: which class each tree belongs to
            `-- trees      7,500 trees, each stored as parallel arrays
```

7,500 trees = 500 rounds × 15 classes. Tree 0 belongs to Benign, tree 1 to Bot, and so on; the `tree_info` list records the class of every tree. Prediction uses the first 498 rounds, which is **7,470 trees**.

Each tree is not stored as nested objects but as **arrays indexed by node number**. Node 0 is the root. For node *i*:

Array	Meaning
<code>left_children[i]</code>	node to go to when the answer is yes; -1 means node <i>i</i> is a leaf
<code>right_children[i]</code>	node to go to when the answer is no
<code>split_indices[i]</code>	which of the 70 features the question tests (0 = Dst Port)
<code>split_conditions[i]</code>	the threshold (goes left if feature < threshold); at a leaf, the leaf value
<code>default_left[i]</code>	where a missing value goes (1 = left)
<code>base_weights[i]</code>	the node's output value
<code>loss_changes[i]</code>	how much this split reduced the loss (used for feature importance)
<code>sum_hessian[i]</code>	how much training data reached the node
<code>parents[i]</code>	the node above <i>i</i>

Tree statistic	Value
Total nodes (all 7,500 trees)	876,836
Total leaves	442,168
Nodes per tree	median 31, largest 1,195, smallest 1
Tree depth	median 7, maximum 10 (the <code>max_depth</code> limit)
Example: root of tree 0	is Dst Port < 137? (feature 0, threshold 137)

04 — DATA TYPES AND THE TUNED CLASS WEIGHTS

What goes in, what comes out, and the final adjustment

Item	Type
Saved model	JSON text file (UTF-8), 55.6 MB
Model in Python	<code>xgboost.XGBClassifier</code> (scikit-learn wrapper) around an <code>xgboost Booster</code>
Input	table of <i>n</i> rows × 70 columns, float32 (32-bit decimal numbers)

Item	Type
Thresholds and leaf values	32-bit floating-point numbers
predict_proba() output	float32 array of n rows x 15 columns; each row sums to 1
predict() output	integer class number 0-14; label_classes.json turns it into a name
Class weights	15 decimal numbers in class_weights.json
Supporting files	model_features.json (70 feature names, in order), label_classes.json

Tuned class weights. A rare class is sometimes the right answer but comes a close second to a large class. tune_class_weights.py multiplies each class's probability by a weight before picking the winner. The weights were searched on the validation set only, keeping Benign recall and overall accuracy at 95% or more. A weight above 1 makes the model more willing to choose that class.

Class	Weight
Bot	0.125
Brute Force -Web	90.5097
DDOS attack-LOIC-UDP	256
DoS attacks-Slowloris	4
FTP-BruteForce	16
All other classes	1

Effect on the test set: DDOS attack-LOIC-UDP recall rose from 90.8% to 99.1% and Brute Force -Web from 92.7% to 95.4%, raising the classes at 95% from 8 to 10. SQL Injection fell from 80.0% to 73.3% and DoS attacks-SlowHTTPTest from 22.2% to 0%; both have under 16 test rows, so one flow changes them by 7–11 points.

05 — HOW TESTING WAS DONE

An honest score on data the model never saw

1. The **465,837-row test set** (20% of every class) was set aside before training and never used for training, early stopping or weight tuning.
2. evaluate_final.py rebuilds the same split (fixed seed 42), loads the saved model and class weights, and computes the 15 probabilities for every test flow.
3. Each probability is multiplied by its class weight; the largest wins.
4. Every prediction is compared with the true label. The counts go into the confusion matrix, and precision, recall, F1 and accuracy are calculated from it.

Speed: predicting all 465,837 test flows took 8.2 seconds, about **56,802 flows per second** on this laptop.

06 — WHAT THE MEASURES MEAN

Accuracy, precision, recall and F1, with this model's numbers

For any one class, every test flow falls into one of three useful groups:

- **True positive (TP):** really this class, and the model said this class. A correct catch.
- **False negative (FN):** really this class, but the model said something else. A missed attack.
- **False positive (FP):** really something else, but the model said this class. A false alarm.

Measure	Formula	Meaning and value
Accuracy	<code>correct predictions / all predictions</code>	Of all 465,837 test flows, how many got the right label. 445,806 / 465,837 = 95.7% .

Measure	Formula	Meaning and value
Recall	$TP / (TP + FN)$	Of the real attacks of a class, how many were caught. This is the "95 out of 100 attacks" measure, also called detection rate. It is the per-attack accuracy used in this report.
Precision	$TP / (TP + FP)$	Of the flows the model called this class, how many really were. Low precision means many false alarms.
F1 score	$2 \times P \times R / (P + R)$	One number that balances precision and recall. It is high only when both are high.
Macro average	plain mean over the 15 classes	Every class counts equally, so tiny classes pull it down. Macro F1 = 0.7918 .
Weighted average	mean weighted by test rows per class	Large classes count more, so it is close to accuracy. Weighted F1 = 0.9558 .

Worked example: Infiltration.

```

TP = 12,177   FN = 11,343   FP = 8,402
recall    = 12,177 / (12,177 + 11,343) = 0.5177   -> 52 of 100 caught
precision = 12,177 / (12,177 + 8,402)  = 0.5917   -> 59 of 100 alarms real
F1        = 2 x 0.5917 x 0.5177 / (0.5917 + 0.5177) = 0.5523

```

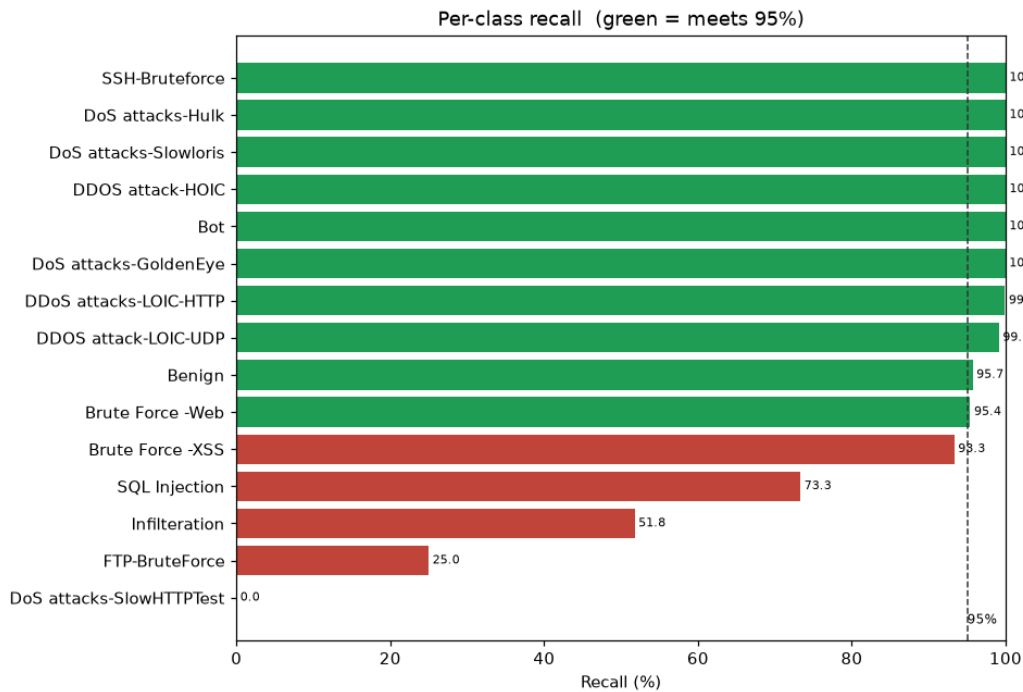
Why per-class accuracy is not used. Counted per class (this class versus everything else), accuracy is above 99% for almost every class, because most flows are correctly "not this class". That hides missed attacks, so recall is the per-attack measure here.

07 — RESULTS FOR EVERY CLASS

Test set, after tuning (sorted by number of test rows)

Class	Test rows	TP	FN	FP	Precision	Recall	F1
Benign	199,922	191,362	8,560	11,349	94.4%	95.7%	0.951
DDoS attacks-LOIC-HTTP	115,073	114,981	92	3	100.0%	99.9%	1.000
DDoS attack-HOIC	39,772	39,772	0	0	100.0%	100.0%	1.000
DoS attacks-Hulk	29,040	29,040	0	0	100.0%	100.0%	1.000
Bot	28,907	28,903	4	0	100.0%	100.0%	1.000
Infiltration	23,520	12,177	11,343	8,402	59.2%	51.8%	0.552
SSH-Bruteforce	18,808	18,808	0	0	100.0%	100.0%	1.000
DoS attacks-GoldenEye	8,281	8,279	2	2	100.0%	100.0%	1.000
DoS attacks-Slowloris	1,982	1,982	0	2	99.9%	100.0%	0.999
DDoS attack-LOIC-UDP	346	343	3	91	79.0%	99.1%	0.879
Brute Force -Web	109	104	5	161	39.2%	95.4%	0.556
Brute Force -XSS	45	42	3	2	95.5%	93.3%	0.944
SQL Injection	15	11	4	1	91.7%	73.3%	0.815
DoS attacks-SlowHTTPTest	9	0	9	6	0.0%	0.0%	0.000
FTP-BruteForce	8	2	6	12	14.3%	25.0%	0.182
Overall	465,837	445,806			95.5%	95.7%	0.956

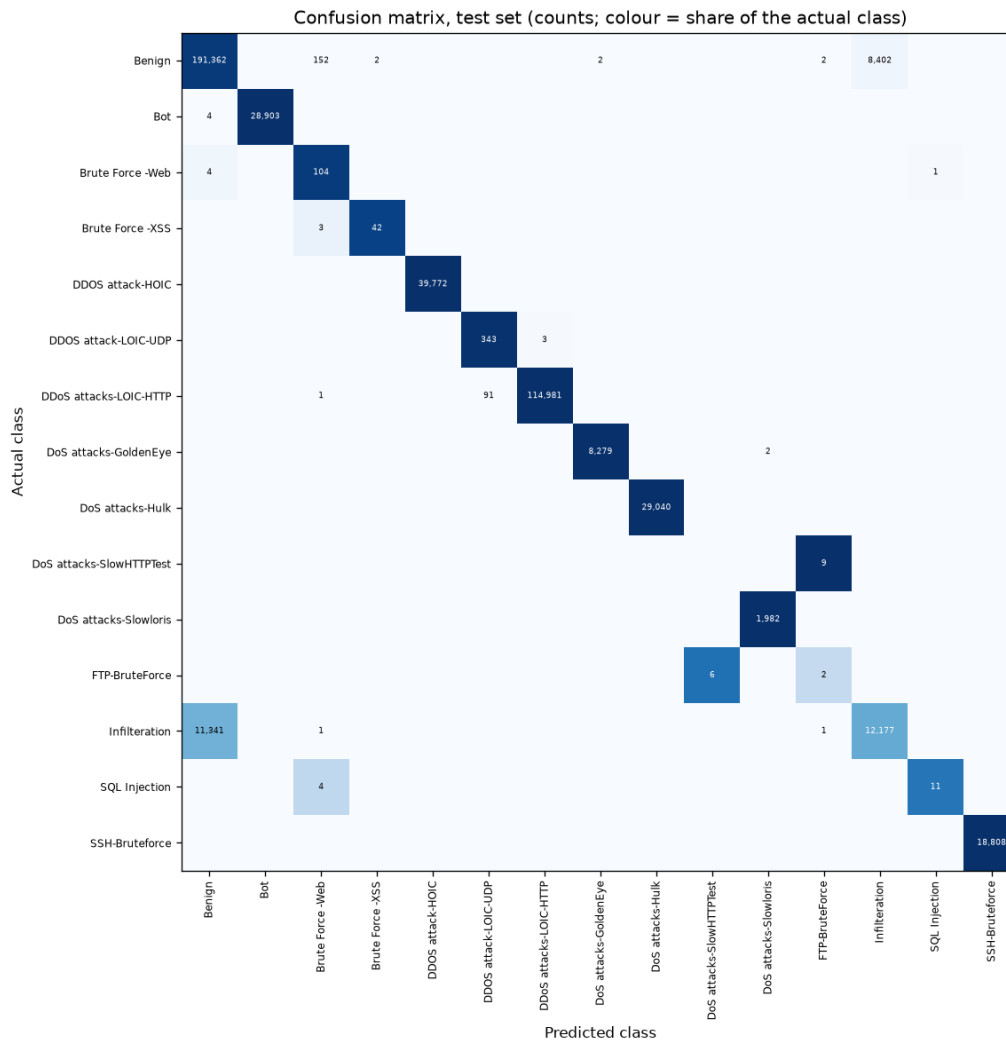
Overall row: the recall column shows overall accuracy; precision and F1 are weighted averages. Green = at least 95% of that class's attacks caught.



08 — THE CONFUSION MATRIX

Where every test flow ended up

The confusion matrix is a 15 × 15 table. **Each row is the true class** and **each column is the predicted class**. A cell holds how many test flows of that true class got that prediction. The diagonal (top-left to bottom-right) holds the correct predictions; everything off the diagonal is a mistake. A row adds up to the class's test rows; a column adds up to how often the model chose that class.



The diagonal adds up to 445,806 correct flows; the other 20,031 cells are mistakes.

The main mistakes

True class	Predicted as	Flows	Why
Infiltration	Benign	11,341	Infiltration flows come from a compromised inside machine behaving almost like a normal user, so many look Benign.
Benign	Infiltration	8,402	The same overlap in the other direction: normal flows that resemble infiltration.
Benign	Brute Force -Web	152	Brute Force -Web has a weight of 90.5, so borderline web flows are pushed to it; this is why its precision is only 39%.

True class	Predicted as	Flows	Why
DDoS attacks-LOIC-HTTP	DDoS attack-LOIC-UDP	91	LOIC-UDP's weight of 256 pulls a few LOIC-HTTP flows across (0.08% of LOIC-HTTP).
DoS attacks-SlowHTTPTest	FTP-BruteForce	9	In this dataset these two classes produce almost identical flows, and only 43 and 39 rows remain after duplicate removal.
FTP-BruteForce	DoS attacks-SlowHTTPTest	6	The same pair, the other way round.

Benign and Infiltration account for 19,743 of the 20,031 mistakes (98.6%). Outside that pair, the model makes 288 mistakes on 465,837 flows.

09 — CONCLUSION

What the model can and cannot do

- **Overall target met.** 95.7% of all test flows are labelled correctly (target 95%).
- **10 of 15 classes meet the 95% recall target.** Bot, SSH-BruteForce, DoS-Hulk, DDoS-HOIC, DDoS-LOIC-HTTP, DDoS-LOIC-UDP, DoS-GoldenEye, DoS-Slowloris, Brute Force -Web and Benign are caught 95.4–100% of the time.
- **Infiltration (51.8%)** is the real weakness. Its flows look like normal traffic, so no tuning of this model can separate them without flooding Benign with false alarms.
- **Four small classes miss the target.** Brute Force -XSS (93.3%, 45 test rows), SQL Injection (73.3%, 15), FTP-BruteForce (25.0%, 8) and DoS attacks-SlowHTTPTest (0%, 9). After duplicate removal they have 39–226 rows in total, too few to learn from or to measure reliably.
- **Possible next steps** (not done yet): merge the small web attacks into one "Web attack" class, drop FTP-BruteForce and SlowHTTPTest from this model, or keep all 15 classes as they are.

Scripts: all_attacks/train_all_attacks.py, tune_class_weights.py, evaluate_final.py. Numbers are from final_metrics.json and confusion_matrix_final.csv. Compiled 2026-09-29.